

Actividad práctica — Tema 1: Introducción a la Arquitectura de Software y aprendizajes previos

1. Exploración conceptual

- ¿Qué es arquitectura de Software?

Podemos definir la arquitectura de software como la estructura general y fundamental de un sistema, esto incluye sus componentes principales, las relaciones entre ellos y las decisiones que guían su diseño y evolución. La arquitectura de software se considera una vista de alto nivel ya que no detalla en la programación, si no que entra en como esta organizado, cómo se comunica y cómo asegura que cumpla tanto lo que el sistema debe hacer. Hay que tener en cuenta que con una buena implementación de la arquitectura de software se puede anticipar problemas, mantener la calidad del software y asegurar que el sistema cumpla sus objetivos. La arquitectura se debe incluir desde proyectos pequeños hasta lo mas grandes, ya que tiene beneficios clave para el desarrollo como:

Escalabilidad: una buena arquitectura permite crecer sin comprometer la estabilidad.

Flexibilidad: facilita adaptar el sistema a nuevas necesidades.

Comunicación: sirve como lenguaje común entre los diferentes roles del proyecto.

Calidad: asegura que el sistema cumpla estándares técnicos y de negocio.

- ¿Cómo se diferencia la arquitectura de software del diseño de software?

Los dos conceptos se relacionan, pero la arquitectura y el diseño abordan distintos niveles del desarrollo:

La arquitectura de software se centra en la visión general del sistema, definiendo cuáles son los componentes principales, cómo se relacionan entre sí y por qué se organiza de esa manera. Responde al “qué” y al “por qué”, y sus decisiones tienen un impacto a largo plazo, ya que establecen la estructura base sobre la cual se construirá todo el sistema. En cambio, el diseño de software se ocupa de los detalles específicos de la implementación. Define cómo se construyen internamente esos componentes mediante clases, estructuras de datos, algoritmos o interfaces. Responde al “cómo” y su impacto es más inmediato, ya que guía el trabajo técnico del día a día.

- Cuadro comparativo arquitectura de software del diseño de software

ARQUITECTURA	DISEÑO
Busca garantizar escalabilidad, flexibilidad y calidad	Se centra en la lógica interna de cada módulo
Se representa con diagramas de arquitectura y modelos de alto nivel	Se representa con diagramas de clases, secuencia o algoritmos
Responde al "qué" y "por qué"	Impacto inmediato o táctico
Se centra en la estructura global del sistema	Responde al "cómo"
Define componentes, módulos, estilos	Busca lograr eficiencia y correctitud en el código

- por qué es importante no confundir estos dos enfoques durante el desarrollo de un proyecto real.

Es importante no confundir estos dos enfoques ya que cada uno cumple un papel distinto dentro del desarrollo. Si se confunden, se corre el riesgo de tomar decisiones de corto plazo que comprometan la estabilidad futura del sistema, o de intentar resolver detalles técnicos sin una base sólida que los soporte. En un proyecto real, esto puede traducirse en sistemas difíciles de mantener, costosos de escalar o que no cumplen con los objetivos del negocio.

2. Análisis aplicado

- ¿Qué hace un arquitecto de software dentro de un equipo técnico?
Un arquitecto de software dentro de un equipo técnico tiene la responsabilidad de guiar la construcción del sistema desde una visión global. Debe asegurar que el software sea escalable, seguro, mantenible y alineado con los objetivos del negocio.
Algunas funciones son:
 - Definir la estructura del sistema.
 - Asegurar calidad técnica.
 - Guiar al equipo.
 - Documentar la solución.
 - Anticipar riesgos y mantener la coherencia del sistema.
- ¿Qué habilidades o conocimientos debe tener este perfil profesional?
 - Dominio de patrones y estilos arquitectónicos.
 - Conocimiento sólido en lenguajes de programación y frameworks modernos.
 - Experiencia en bases de datos relacionales y no relacionales.
 - Conocimientos de seguridad, escalabilidad y rendimiento.

- E. Manejo de metodologías de desarrollo (ágiles, DevOps, integración continua).
 - F. Experiencia en infraestructura y cloud computing (AWS, Azure, GCP).
 - G. Liderazgo técnico
 - H. Comunicación clara
 - I. Resolución de problemas
 - J. Adaptabilidad.
- ¿Qué consecuencias puede haber si un proyecto no considera decisiones arquitectónicas desde el inicio?
Si un proyecto no toma en cuenta decisiones arquitectónicas desde el inicio, pueden aparecer varios problemas que afectan el desarrollo y el resultado como:
 - A. Falta de escalabilidad.
 - B. Problemas de rendimiento.
 - C. Altos costos de mantenimiento.
 - D. Dificultad para integrar nuevas tecnologías.
 - E. Riesgos de seguridad.
 - F. Retrasos en el proyecto.
 - G. Calidad deficiente del producto.

3. Caso práctico

Actividad

Imagina que estás diseñando una aplicación para gestionar préstamos de libros en una biblioteca universitaria.

- Menciona tres decisiones arquitectónicas que deberían tomarse desde el inicio del proyecto (ej: tecnologías, estructura de componentes, tipos de base de datos, etc.).
- Reflexiona: ¿esas decisiones son de diseño o de arquitectura? Justifica.
- **Estilo y estructura del sistema**
Se debe decidir si se arranca con un monolito modular (todo en una sola aplicación bien separada por módulos) o con microservicios (servicios independientes: catálogo, préstamos, usuarios, notificaciones, etc.).
Por qué importa: esta elección afecta la escalabilidad, el despliegue, la complejidad operativa y el tiempo de desarrollo. Una mala decisión puede volver lento el desarrollo o encarecer el mantenimiento.
Consecuencias típicas: un monolito es más simple al inicio y tiene menos sobrecarga operacional; los microservicios permiten escalar mejor y aislar fallos, pero requieren mayor esfuerzo en DevOps.

¿Arquitectura o diseño? **Arquitectura**. Es una decisión estratégica que define la organización global del sistema.

- **Modelo de persistencia y consistencia (RDBMS vs NoSQL / transaccionalidad)**

Qué Se debe decidir: usar una base de datos relacional (ACID: ideal para préstamos, transacciones y consistencia) o alguna NoSQL (útil para catálogo de gran volumen o búsquedas rápidas). También definir el nivel de consistencia (fuerte vs eventual).

Por qué importa: los préstamos implican transacciones (evitar el doble préstamo de un mismo ejemplar), historial académico y auditoría; una mala elección puede provocar inconsistencias y problemas legales u operativos. Consecuencias típicas: un RDBMS facilita la integridad y consultas complejas; NoSQL puede acelerar la búsqueda y el escalado del catálogo, pero complica las transacciones.

¿Arquitectura o diseño? **Arquitectura (principalmente)**. Es una decisión de alto impacto que condiciona integridad, rendimiento y operaciones. Nota: la normalización de tablas, índices y esquemas concretos son decisiones de diseño.

- **Infraestructura y despliegue (on-prem / cloud, contenedores, CI/CD y alta disponibilidad)**

Se debe decidir: si el sistema se desplegará en nube pública (AWS/GCP/Azure) o en infraestructura local, si se usarán contenedores (Docker + Kubernetes) o simplemente máquinas virtuales, y cómo será el pipeline de CI/CD y los backups.

Por qué importa: esta decisión define costos, tolerancia a fallos, automatización, tiempos de entrega y capacidad para responder a picos (por ejemplo, al inicio de semestre). También afecta el monitoreo y la recuperación ante desastres.

Consecuencias típicas: nube + contenedores = escalado y automatización; infraestructura local = mayor control y posible ahorro, pero menor elasticidad.

¿Arquitectura o diseño? **Arquitectura**. Es una decisión estratégica sobre cómo se entrega y opera el sistema. Los scripts de despliegue concretos y la configuración fina corresponden a decisiones de diseño u operación.

